Phishing Kit Hunter

A simple explanation of the whole project

What the project is about, why it was built, how it was built step by step, what each file does, and what the project can do now.

Contents

1. What is this project about?

2. Important words in simple language

3. The big picture: how the project works

4. Safety rules we followed

5. Setting up the machine (what we did and why)

6. The project folder, files and database

7. Step by step: what we built, how, and why

8. Results we got

9. How the detection works (with an example)

10. What the project does now, and how to run it

11. Limitations (what the project cannot do)

12. Future improvements

13. Questions you may be asked, with simple answers

14. Command cheat sheet

1. What is this project about?

1.1 The problem: phishing

Phishing is when criminals make a fake website that looks exactly like a real one, for example a fake Microsoft login page, a fake Roblox page, or a fake bank page. People are tricked into typing their username and password into the fake page, and the criminals steal them.

Making a fake website from nothing takes time, so criminals use a ready-made package. This package is called a **phishing kit**. A kit is a bundle of files: HTML pages, images, CSS, JavaScript, and usually a small script that sends the stolen passwords to the criminal (for example to a Telegram chat or an email address). The criminal uploads the kit to a hacked website or a free hosting service, and the fake login page is live within minutes.

The key point is this: **one kit is used again and again**. The same kit can appear on hundreds of different website addresses (URLs). The addresses look different, but the pages underneath are almost the same.

1.2 Why detect kits and not just URLs?

Normally, security teams block a bad URL after someone reports it. But the criminal simply uploads the same kit to a new URL and the attack starts again. If we can recognise the kit itself, we can catch the new copies too, even if we have never seen that URL before. That is the idea of this project.

1.3 What my project does, in one paragraph

Phishing Kit Hunter automatically (1) collects phishing URLs from public feeds (OpenPhish, PhishTank and Phishing.Database), (2) safely downloads the page of each URL, (3) measures many features of each page, (4) groups pages that come from the same kit, (5) writes a detection rule for each kit, (6) tests how good those rules are on pages it has not seen before and on normal websites, and (7) shows all results in statistics, a chart and a dashboard. There is also a small tool that checks any new URL against the rules.

1.4 Project objectives

- Collect live phishing URLs from public feeds.
- Analyse the pages and find features that are common inside one kit.
- Group pages into kits (clustering).
- Create detection rules for each kit and explain how they work.
- Measure how accurate the rules are (detection rate and false positives).
- Present the results clearly (statistics, chart, dashboard, report).

2. Important words in simple language

Word	Simple meaning
Phishing	A fake website or message that tricks people into giving passwords or money.
Phishing kit	The ready-made set of files criminals use to build a fake login page.
URL	The web address of a page, for example https://example.com/login.
Feed	A public list of bad URLs that is updated all the time. We used OpenPhish, PhishTank and Phishing.Database.
HTML	The text code of a web page. We read the HTML to study the page without opening it in a browser.
Feature / fingerprint	A small fact about a page that helps us compare it, such as its title, the names of its form fields, or the files it loads.
Hash	A short code made from some text. The same text always gives the same hash. Two pages with the same structure hash have exactly the same layout.
TLSH	A special hash for similarity. A normal hash changes completely if one letter changes, but TLSH gives a small distance number for pages that are almost the same. Small distance means similar pages.
Cluster	A group of similar things. Here, one cluster means one phishing kit.
Exfiltration endpoint	The place where the stolen data is sent, such as a Telegram bot, a Discord webhook, a Formspree form or another website.
Obfuscation	Hiding code so it is hard to read, for example with eval(), atob() or hex codes.
YARA	A standard tool and rule language used by security teams to find files that contain certain text patterns. We use it for our detection rules.
Detection rate	Out of all the kit pages we tested, the percentage the rules caught.
False positive	A normal, safe page that the rules wrongly call phishing. We want this to be zero or very low.
Benign page	A normal, safe page (for example the Google or GitHub login page). We use them to test false positives.
Kali Linux	A Linux system that comes with security tools. We run it inside VirtualBox.
Virtual machine (VM)	A computer running inside your computer. If something goes wrong, it does not harm your real computer.
Virtual environment (venv)	A private folder of Python libraries for one project, so it does not mix with other projects.
SQLite	A small database stored in one file (db/phish.db). We use it to store URLs, page details and kit groups.
Git	A tool that saves versions of your project, like save points.

3. The big picture: how the project works

The project is a pipeline. Each step takes the output of the previous step and passes its own output to the next one.

```
Public feeds (OpenPhish, PhishTank, Phishing.Database)
           |
           v
1. COLLECT  - save the URLs in the database
           |
           v
2. FETCH    - download the page of each URL safely
           |
           v
3. EXTRACT  - measure features of each page
           |
           v
4. CLUSTER  - group similar pages = one kit
           |
           v
5. RULES    - write a YARA rule for each kit
           |
           v
6. TEST     - try the rules on unseen phishing pages and on normal websites
           |
           v
7. REPORT   - statistics, chart and dashboard

Extra tool: DETECT - give it any URL and it tells you if it matches a known kit
```

4. Safety rules we followed

We are handling real phishing pages, so safety is important. This is also worth writing in your report.

- Everything runs inside the Kali virtual machine, not on the real computer.
- We never open the phishing URLs in a browser. Our Python script only downloads the page text. It does not run any JavaScript, so nothing can attack us.
- We never type any username or password into any phishing page.
- We only use public feeds that are meant for research and defence.
- The fetch script limits each download to 2 MB and uses a 12 second timeout.
- Links shown in the dashboard are written in a defanged way (hxxp and [.] instead of http and dots) so nobody clicks them by mistake.
- We saved the work often with Git, and shut the VM down properly with poweroff.

5. Setting up the machine (what we did and why)

5.1 Kali Linux in VirtualBox

You already had Kali Linux running in Oracle VirtualBox (the machine is named Kali-Linux in VirtualBox). **Why:** a virtual machine is a safe, separate computer. Kali already includes many security tools, and the project can be repeated on any computer that has VirtualBox.

5.2 Installing system packages

```
sudo apt install -y python3-pip python3-venv python3-dev build-essential cmake libssl-dev
   libmagic-dev git sqlite3 curl wget nano tree unzip cron
```

What this does: it installs the tools the project needs. **Why:**

- python3-pip, python3-venv, python3-dev: Python, its library installer, and virtual environments.
- build-essential, cmake, libssl-dev, libmagic-dev: tools needed to build some Python libraries (TLSH and YARA need to be compiled).
- git: saves versions of the project.
- sqlite3: lets us look inside the database from the terminal.
- curl and wget: download files and test the feeds from the terminal.
- nano: a simple text editor. tree: shows the folder structure. unzip: opens zip files. cron: can run the project automatically every day (a future improvement).

5.3 Creating the folder structure

```
mkdir -p phish-kit-hunter/{data/raw_feeds,data/pages,data/benign,db,src,rules,reports,logs}
```

Why: a clean structure keeps raw data, the database, the code, the rules and the reports in separate places. Section 6 explains each folder.

5.4 Creating the virtual environment

```
python3 -m venv venv
source venv/bin/activate
```

Why: the libraries are installed only for this project. When the environment is active you see (venv) at the start of the terminal line. You must run *source venv/bin/activate* again in every new terminal.

5.5 Installing the Python libraries

Library	**What we use it for**
requests	Downloads feeds and web pages.
beautifulsoup4 and lxml	Reads the HTML of a page and finds forms, input fields, scripts and the title.
tldextract	Splits a URL into domain and ending (TLD) correctly, for example .com or .dev.
python-tlsh	Makes the TLSH similarity hash of each page.
yara-python	Lets Python compile and run YARA rules.
matplotlib	Draws the chart of pages per kit.

Library	What we use it for
pandas, numpy, scikit-learn, networkx, ipwhois	Installed for analysis ideas (data tables, machine learning, graphs, hosting owner lookup). The final pipeline does not need them, so they are available for future improvements.

We also wrote a small file, src/test_setup.py, that tries to import every library and prints OK or FAILED. This is a quick way to check that the installation is correct.

5.6 Testing the feed and setting up Git

We tested the OpenPhish feed with *curl -sL https://openphish.com/feed.txt | head -5*. If five URLs appear, the internet and the feed are working. Then we ran *git init*, created a .gitignore file and made the first commit. After that we committed again at the end of every work session. **Why:** if a file is broken or lost, we can go back to an earlier version.

6. The project folder, files and database

6.1 Folders

Folder	What is inside
data/raw_feeds	A saved copy of each feed download, with the date and time in the file name (proof of where the URLs came from).
data/pages	The downloaded HTML page of each phishing URL. The file name is the SHA-256 hash of the page, so identical pages are saved only once.
data/benign	HTML pages of normal websites, used to test false positives.
db	phish.db, the SQLite database.
src	All the Python scripts.
rules	kits.yar, the generated YARA rules.
reports	evaluation.txt, stats.txt, kit_sizes.png and dashboard.html.
logs	Reserved for log files in future.

6.2 The scripts in src

File	What it does
config.py	Keeps the folder paths and settings in one place (for example MIN_TAGS = 15).
db.py	Creates the database tables and has helper functions to load data.
collect.py	Step 1: downloads URLs from the feeds and stores them.
fetch.py	Step 2: downloads the pages.
extract.py	Step 3: measures the features of each page.
cluster.py	Step 4: groups pages into kits.
rules.py	Step 5: writes YARA rules from the kits.
benign_fetch.py	Downloads normal websites for testing.

File	What it does
evaluate.py	Step 6: tests the rules and calculates detection rate and false positives.
detect.py	Live demo: checks one URL against all the rules.
report.py	Step 7: makes statistics and the chart.
dashboard.py	Builds the HTML dashboard from all the results.

6.3 The database tables

Table	What it stores
urls	Every collected URL, its domain, the feed it came from, the brand name (PhishTank only), the time it was first seen, and a status (new, fetched, dead, failed, nothtml).
pages	For each downloaded page: final URL after redirects, HTTP status, redirect chain, server name, the saved file name, SHA-256, IP address, and whether the web folder was open.
features	The measured features of each page (see step 3 in section 7).
clusters	Which kit each page belongs to (KIT-001, KIT-002 and so on, or single).
benign	The normal websites used for testing.
hosts	Created for hosting owner details (ASN, organisation, country). It is not filled yet; it is planned for future work.

7. Step by step: what we built, how, and why

Step 1: Collect the URLs (collect.py)

What

The script downloads three public feeds and stores every URL in the database.

- **OpenPhish:** a plain text list with one URL per line.
- **PhishTank:** a compressed CSV file (online-valid.csv.gz). It also gives the brand being copied, for example Microsoft or PayPal. We keep the newest 500 entries.
- **Phishing.Database:** a list of active phishing links from GitHub. It is used with the --extra option and we keep the last 500.

How

- Each download is saved in data/raw_feeds with the date and time.
- Each URL is cleaned. Anything that does not start with http or https is thrown away.
- tldextract finds the main domain of each URL.
- The command INSERT OR IGNORE puts the URL in the database. The URL column is unique, so running the script again never creates duplicates.

Why

Three feeds give more variety than one, and saving the raw copies proves where the data came from. Because duplicates are ignored, we can run collect.py every day and only new URLs are added.

Result

1300 URLs in total: OpenPhish 300, PhishTank 500 and Phishing.Database 500.

Step 2: Fetch the pages (fetch.py)

What

For each new URL the script downloads the page so we can study it.

How

- It uses 10 workers at the same time (ThreadPoolExecutor), so many URLs are fetched quickly.
- It sends a normal browser User-Agent, because some phishing pages hide from scripts.
- It follows redirects and records the full redirect chain.
- It does not stop for bad SSL certificates (verify=False), because many phishing sites have broken certificates.
- It reads at most 2 MB and waits at most 12 seconds.
- Status codes of 400 or above are marked dead. Pages that are not text or HTML are marked nothtml. Network errors are marked failed.
- For a good page it saves the HTML under the SHA-256 name, and stores the IP address, server header, X-Powered-By header and redirect chain.
- It checks if the page is an open directory (the text Index of /). If so, it looks for archive links such as .zip or .rar, because that could be the kit itself.
- We changed the order so that OpenPhish and PhishTank URLs are tried before Phishing.Database URLs.

Why

We need the real page content to compare pages. Phishing sites die very quickly, so many downloads fail. That is normal and expected, and it is a good point to mention in the report.

Result

237 pages were downloaded. 146 URLs were dead, 316 failed (no connection or error), 1 was not HTML, and 600 URLs were not tried yet (they are still marked new and can be fetched later).

Step 3: Extract features (extract.py)

What

For every downloaded page, the script measures facts (features) that help us compare pages.

Feature	Meaning and why it helps
title, number of tags	The page title and the size of the page. Pages with fewer than 15 tags are almost empty (404 or blank), so they are not clustered.
forms, form actions	Where each form sends its data. Kits often use the same script name, for example post.php.
input names, has_password	The names of the form fields (email, passwd, ...) and whether the page has a password box. A password box is the sign of a credential-stealing page.
local files	The scripts, style files and images loaded from the same site. A kit usually has the same file names everywhere.

Feature	Meaning and why it helps
external domains	Other websites the page loads files from.
has_obfuscation	Yes if the code uses eval(), atob(), unescape(), fromCharCode or hex codes to hide itself.
exfil hint	Where stolen data goes: a Telegram bot ID, a Discord webhook ID, a Formspree ID, or a form action that points to another website. If two pages send data to the same Telegram bot, they almost certainly belong to the same criminal and kit.
structure hash	A hash of the order of all HTML tags. Same hash means the same page layout.
resource hash	A hash of the sorted list of local file names.
TLSH	A similarity hash of the whole page. Used to find pages that are similar but not identical.
URL features	URL length, number of dots and hyphens, the TLD (.com, .dev) and the file name in the path.
HTML comments	The first five comments in the page. Kit authors sometimes leave signatures there.

Why

Looking at pages one by one is impossible with hundreds of pages. Features turn each page into a small fingerprint that the computer can compare.

Result

237 pages were analysed (the same number that were fetched). 81 had a password field, 52 used obfuscated JavaScript, and 12 had a data-stealing endpoint.

Step 4: Cluster the pages into kits (cluster.py)

What

The script decides which pages belong to the same kit. Each group gets a name such as KIT-001.

How

Two pages are linked if any one of these is true:

- they have the same structure hash (same layout),
- they have the same resource hash (same set of files),
- they have the same exfiltration hint (same Telegram bot, webhook or form target),
- their TLSH distance is 70 or less (the pages are very similar).

All pages that are linked directly or through other pages end up in the same group. This method is called union-find. Groups with 2 or more pages become KIT-001, KIT-002 and so on, starting with the biggest. Pages that match nothing are marked single. The value 70 (TLSH_MAX in the file) can be changed: a lower number gives stricter groups, and a higher number gives looser groups.

Why

This is the core of the project. If many different URLs have the same kit, the group shows how big and active that kit is, and gives us enough pages to build a reliable rule.

Result

200 pages were clustered (37 were skipped because they were almost empty). 34 kits were found. 121 pages belong to a kit and 79 pages stayed single. The largest groups were:

Kit	Pages	Common page title
KIT-001	20	I (a very small page; it may be a loader or redirect page)
KIT-002	10	Account Verification
KIT-003	7	Security verification - Microsoft account
KIT-004	5	ZLIB_TIKTOKER - Roblox
KIT-005	5	Telstra
KIT-006	4	Telkomsa
KIT-007	4	Meta Verified - Rewards for you
KIT-008	4	ebay Austin - Registration
KIT-009	4	Chineso2 - Roblox
KIT-010 to KIT-015	3 each	Webmail login, Meta Page Appeal, Sign in to your account, and others

The titles inside each group match, which shows the grouping works. The brand column was empty because only PhishTank gives a brand name.

Step 5: Make the detection rules (rules.py)

What

For each kit, the script writes a YARA rule that can recognise any page of that kit.

How

- Only 70 percent of the pages are used to build the rules (the pages whose database id ends in 0 to 6). The other 30 percent are kept hidden for testing, so the test is fair.
- A kit needs at least 3 training pages to get a rule.
- The script finds the things that appear in at least 80 percent of the kit's pages: local file names, the page title, the data-stealing endpoint, and the form input names.
- Common library files (jquery, bootstrap, analytics, font-awesome, favicon and similar) are ignored because they appear on normal sites too.
- File names, the title and the endpoint are called distinctive strings. The input names are used only as a second check.
- The rule condition is: at least 60 percent of the distinctive strings must be present, AND at least one of the form input names.
- The rules are saved in rules/kits.yar and the script checks that YARA can compile them.

Why

A rule lets any security tool find the kit without our Python code. YARA is the industry standard, so the result can be used in real security systems. Requiring both distinctive strings and a form field makes the rule strict enough to avoid wrongly flagging normal websites.

Step 6: Collect normal pages and test the rules (benign_fetch.py and evaluate.py)

What

We must prove two things: the rules catch kit pages they have not seen before, and they do not flag normal sites.

How

- benign_fetch.py downloaded login and home pages of well-known sites such as Google, Microsoft, GitHub, LinkedIn, Instagram, Netflix, Dropbox, Reddit, Yahoo, Wikipedia, Amazon, Flipkart, Python, Kali, SBI and Discord. 21 pages were saved. Five were skipped or failed (Facebook, PayPal, WordPress and Adobe refused the request, and IRCTC timed out).
- evaluate.py loads the 30 percent of pages that were NOT used to make the rules.
- It scans each of them with the YARA rules and counts how many are detected, and how many are detected as the correct kit.
- It scans every normal page and counts false positives.
- The numbers are printed and saved in reports/evaluation.txt.

Why

A detection rate alone can be misleading, because a rule that flags every page would catch 100 percent. Testing on normal pages shows whether the rules are accurate. Testing on unseen pages shows the rules work on new copies of a kit and not only on the pages used to create them.

Step 7: Detect a live URL (detect.py)

This is the part that shows how the system works in real use. You give it a URL. It downloads the page and scans the content with all the rules. If a rule matches it prints PHISHING KIT DETECTED and the kit name. Otherwise it prints No known kit matched. We tested it with a held-out phishing URL and with python.org.

```
python3 src/detect.py "SUSPICIOUS_URL"
```

Step 8: Statistics, chart and dashboard (report.py and dashboard.py)

report.py counts everything in the database and writes reports/stats.txt. It also draws reports/kit_sizes.png, a bar chart of the biggest kits. dashboard.py combines everything into one web page, reports/dashboard.html. It shows summary cards, the project pipeline, key findings, charts, a searchable list of all kits (with defanged example URLs), the test results, the explanation of how detection works, and the YARA rules. You open it in Firefox.

Step 9: Saving the work with Git

After each stage we ran *git add .* and *git commit -m "message"*. The last commits saved the clustering, rules, evaluation, report and the dashboard.

8. Results we got

8.1 Data collection and fetching

Item	Value
URLs collected	1300 (OpenPhish 300, PhishTank 500, Phishing.Database 500)
Pages fetched live	237
Dead URLs	146
Failed (error or no connection)	316
Not HTML	1
Not tried yet	600
Open directories found	0

8.2 What the pages contain

Item	Value
Pages with a password field	81
Pages with obfuscated JavaScript	52
Pages with a data-stealing endpoint	12
Most common endings (TLDs)	com (80), dev (35), app (33), io (11), cx (9)
Most common hosting IPs	142.250.29.132 (12), 213.136.93.167 (10), 172.66.47.11 (10), 172.66.44.245 (10), 5.175.169.201 (6)

Many phishing pages use endings such as .dev and .app, and several of the IP addresses appear to belong to large platforms such as Google and Cloudflare. This fits the idea that criminals like free or cheap hosting services that are easy to set up and hard to block completely.

8.3 Clustering

Item	Value
Pages used for clustering	200
Kits found	34
Pages that belong to a kit	121 (about 60 percent)
Pages not grouped (single)	79

8.4 Rule testing

The rules were built from 70 percent of the pages and tested on the other 30 percent, which they had never seen. They were also tested on 21 normal websites.

Item	Value
Unseen pages from known kits	37
Detected by any kit rule	16
Detected as the correct kit	15
Detection rate	43.2 percent
Unseen pages that belong to no kit (single)	18 (0 detected, which is expected because no rule exists for them)
Normal websites tested	21
False positives	0 (0.0 percent)

What these numbers mean. The rules caught 16 of the 37 unseen pages from known kits (43.2 percent), and 15 of those 16 were matched to the correct kit. None of the 21 normal websites was flagged, so the rules are strict and reliable but they miss more than half of the unseen kit pages. Two reasons explain the misses. First, many kits are small (3 to 5 pages), so after the 70 percent split a kit may have fewer than 3 training pages and gets no rule at all. Second, a rule needs most of its strings and a matching form field, so a copy of the kit with renamed files will not match. This is the usual trade-off between catching more kits and avoiding false alarms. Possible improvements are a lower minimum number of training pages, a lower commonality threshold, and more pages per kit.

9. How the detection works (with an example)

Think of a kit as a person and the rule as a description of that person: wearing a red cap, carrying a blue bag, and walking with a limp. We do not need to see the face. If most of the details match, it is probably the same person. The rule works the same way:

- The kit pages share some things: unusual file names, the page title, the data-stealing endpoint and certain form field names.
- The rule lists those things as strings.
- A new page is scanned. If at least 60 percent of the distinctive strings are found and one of the form fields is present, the page is flagged as that kit.

This is the shape of a YARA rule produced by rules.py. The file names and numbers below are only an example to show the format. Your real rules are in rules/kits.yar.

```
rule KIT_001
{
    meta:
        description = "Phishing kit cluster KIT-001"
        training_pages = 14
    strings:
        $d1 = "login-style.css" nocase
        $d2 = "verify.js" nocase
        $d3 = "api.telegram.org/bot123456" nocase
        $i1 = /name=["']email["']/ nocase
        $i2 = /name=["']passwd["']/ nocase
```

```
    condition:
        2 of ($d*) and any of ($i*)
}
```

Why this is a good method: the rule uses the same features that the clustering found. Because we built it from what is common to most pages of the kit, a new copy of the same kit will match it even if the URL, the domain and the hosting are all different. The threshold (60 percent) is a balance: lower catches more modified copies but may flag normal pages, higher is stricter but may miss changed kits.

10. What the project does now, and how to run it

10.1 What it can do today

- Collect fresh phishing URLs from three public feeds and keep them in a database without duplicates.
- Download the pages safely and record details such as IP address, redirects and server.
- Measure about 20 features of each page.
- Group pages into kits and show which kits are the biggest.
- Create YARA rules for the kits automatically.
- Test the rules on unseen phishing pages and on normal websites and report detection rate and false positives.
- Check any new URL against the known kits.
- Produce statistics, a chart and an interactive dashboard.

10.2 How to run the whole project again

Open a terminal and run these commands one by one:

```
cd ~/phish-kit-hunter
source venv/bin/activate
python3 src/collect.py --extra      # get new URLs
python3 src/fetch.py 300            # download up to 300 new pages
python3 src/extract.py              # measure features
python3 src/cluster.py              # group into kits
python3 src/rules.py                # write YARA rules
python3 src/benign_fetch.py         # only needed once
python3 src/evaluate.py             # test the rules
python3 src/report.py               # statistics and chart
python3 src/dashboard.py            # build the dashboard
firefox reports/dashboard.html &    # open the dashboard
```

If you run the project again, new pages may change the kits and the rules. Save your final results and screenshots first.

11. Limitations (what the project cannot do)

- Phishing sites die quickly, so only 237 of the 1300 URLs could be downloaded. The sample is small.
- The pages are downloaded as plain HTML. Pages that build their content with JavaScript may look nearly empty to us.
- No open directories were found, so we could not download any real kit ZIP file. We studied the kits only from the pages they create.
- The clustering used all pages, while the rules were tested on a held-out part. This is a small simplification.
- Attackers can change file names or text in their kit, and then a rule may stop matching.
- Only 21 normal websites were used for the false positive test, and many of them load their content with JavaScript. A larger and more varied test set would be better.
- Brand names come only from PhishTank, so the target brand is empty for the other feeds.
- KIT-001 is a very small page (its title is only a single letter), so it may contain several unrelated small pages. The TLSH_MAX setting can be made stricter to separate them.

12. Future improvements

- Run the pipeline every day with cron and track how kits grow over time.
- Fill the hosts table with the hosting owner and country using ipwhois.
- Compare screenshots of the pages to find kits that look the same but have different code.
- Use a larger set of normal sites (for example the Tranco top sites list) to test false positives properly.
- Draw a network graph of kits, domains and IPs with networkx.
- Add machine learning (for example scikit-learn) to score how likely a page is phishing.
- Share the YARA rules with a blocklist or a security tool so others can use them.

13. Questions you may be asked, with simple answers

Q. What is a phishing kit?

A. A ready-made package of files that criminals upload to a website to create a fake login page and steal passwords.

Q. Why did you group pages into kits instead of just blocking URLs?

A. One kit is reused on many URLs. A rule for the kit catches new URLs automatically, while blocking a single URL does not help once the criminal moves to a new address.

Q. Where did you get the data?

A. From the public feeds OpenPhish, PhishTank and Phishing.Database.

Q. Why do you use TLSH and not a normal hash such as MD5 or SHA-256?

A. A normal hash changes completely if even one character changes. TLSH gives a small distance for pages that are almost the same, so it can find slightly modified copies of a kit.

Q. What is YARA?

A. A standard rule language used by security teams to find files that match text patterns.

Q. Why did you split the data into 70 percent and 30 percent?

A. The rules are made from 70 percent. The other 30 percent are unseen pages used to test the rules fairly.

Q. What is a false positive and how did you test it?

A. A safe page wrongly flagged as phishing. We scanned real login and home pages of well-known sites with the rules and counted how many were flagged.

Q. Is it safe to download phishing pages?

A. We worked inside a virtual machine, never opened the pages in a browser, only downloaded the text, and never entered any data.

Q. Why did so many URLs fail to download?

A. Phishing pages are usually taken down within hours or days, so most URLs in the feeds are dead by the time we fetch them.

Q. What are the limitations?

A. A small sample, only static HTML, no kit ZIP files found, a small set of normal pages for testing, and kits can be changed by attackers. Section 11 lists them all.

Q. How can the project be improved?

A. Run it daily, add screenshot comparison, add hosting information, use more normal pages for testing and add machine learning. Section 12 lists these.

14. Command cheat sheet

```
# Start work in a new terminal
cd ~/phish-kit-hunter
source venv/bin/activate

# Check the database
sqlite3 db/phish.db "SELECT COUNT(*) FROM urls;"
sqlite3 db/phish.db "SELECT source, status, COUNT(*) FROM urls GROUP BY source, status;"

# See kits
sqlite3 -header -column db/phish.db \
  "SELECT kit_id, COUNT(*) FROM clusters GROUP BY kit_id ORDER BY 2 DESC LIMIT 10;"

# See the rules and results
head -30 rules/kits.yar
cat reports/evaluation.txt
cat reports/stats.txt

# Open the chart and the dashboard
xdg-open reports/kit_sizes.png
firefox reports/dashboard.html &

# Save your work
git add .
git commit -m "Describe what you changed"

# Shut down the VM properly
sudo poweroff
```